

Resumen

Los mercados de activos digitales mueven cientos de millones anuales y, aun así, carecen de herramientas analíticas y de automatizaciones integradas o de libre uso. La motivación principal de este proyecto nace de esta profunda brecha tecnológica. Actualmente, la gestión de carteras en la mayoría de los mercados se realiza de forma manual o mediante herramientas estáticas, lo que resulta altamente ineficiente y arriesgado.

El propósito principal de este TFM es evaluar si un ecosistema descentralizado de agentes de inteligencia artificial puede superar las limitaciones humanas y generar rendimientos estables en mercados de alta fricción. Se opta por una arquitectura multiagente de aprendizaje por refuerzo para descentralizar la toma de decisiones y reducir la complejidad computacional del mercado. El objetivo es que los agentes aprendan estrategias de inversión óptimas que maximicen el saldo total de la cartera.

La metodología desarrollada en el TFM se aplica al ámbito de la economía virtual de Counter-Strike 2, un popular videojuego que cuenta con un ecosistema financiero integrado de alta frecuencia y liquidez.

Palabras clave: Mercados de activos digitales, toma de decisiones, arquitectura multiagente, aprendizaje por refuerzo.

Resum

Els mercats d'actius digitals mouen centenars de milions anuals i, tot i això, encara no disposen d'eines analítiques ni d'automatitzacions integrades o de lliure ús. La motivació principal d'aquest projecte naix d'aquesta profunda bretxa tecnològica. Actualment, la gestió de carteres en la majoria dels mercats es realitza de forma manual o mitjançant eines estàtiques, la qual cosa resulta altament ineficient i arriscada.

El propòsit principal d'aquest TFM és avaluar si un ecosistema descentralitzat d'agents d'intel·ligència artificial pot superar les limitacions humanes i generar rendiments estables en mercats d'alta fricció. S'opta per una arquitectura multiagent d'aprenentatge per reforç per descentralitzar la presa de decisions i reduir la complexitat computacional del mercat. L'objectiu és que els agents aprenguen estratègies d'inversió òptimes que maximitzen el saldo total de la cartera.

La metodologia desenvolupada en el TFM s'aplica a l'àmbit de l'economia virtual de Counter-Strike 2, un videojoc popular que compta amb un ecosistema financer integrat d'alta freqüència i liquiditat.

Paraules clau: Mercats d'actius digitals, presa de decisions, arquitectura multiagent, aprenentatge per reforç.

Abstract

Digital asset markets move hundreds of millions annually and yet still lack integrated or freely available analytical tools and automation systems. The main motivation of this project stems from this deep technological gap. At present, portfolio management in most markets is carried out manually or through static tools, which makes it highly inefficient and risky.

The main purpose of this master's thesis is to evaluate whether a decentralized ecosystem of artificial intelligence agents can overcome human limitations and generate stable returns in high-friction markets. A multi-agent reinforcement learning architecture is chosen to decentralize decision-making and reduce the computational complexity of the market. The goal is for the agents to learn optimal investment strategies that maximize the total portfolio balance.

The methodology developed in this master's thesis is applied to the virtual economy of Counter-Strike 2, a popular video game with an integrated financial ecosystem characterized by high frequency and liquidity.

Key words: Digital asset markets, decision-making, multi-agent architecture, reinforcement learning.

Índice general

Índice de figuras	7
Índice de tablas	8
Lista de algoritmos	9
Lista de acrónimos	10
1 Introducción	12
1.1 Motivación	12
1.2 Planteamiento del problema	12
1.3 Objetivos	13
1.4 Alcance y restricciones	13
1.5 Metodología de desarrollo	14
1.6 Estructura del documento	14
2 Marco teórico y estado del arte	16
2.1 Mercados de activos digitales	16
2.1.1 Activos virtuales en Counter-Strike 2	16
2.1.2 Fricciones de mercado	16
2.2 Aprendizaje por refuerzo	17
2.2.1 Procesos de decisión de Markov	17
2.2.2 Política, retorno y exploración	17
2.3 Aprendizaje por refuerzo multiagente	18
2.3.1 Cooperación y coordinación entre agentes	18
2.3.2 Entrenamiento centralizado y ejecución descentralizada	18
2.4 Toma de decisiones en carteras	19
2.4.1 Rentabilidad y riesgo	19
2.4.2 Liquidez y costes de transacción	19
2.5 Aplicaciones relacionadas	20
2.6 Tecnologías utilizadas	20
2.6.1 Herramientas de desarrollo	20
2.6.2 Adquisición y tratamiento de datos	21
2.6.3 Persistencia y modelo de datos	21
2.6.4 Predicción, simulación y entrenamiento multiagente	21
2.6.5 Interfaz y operación local	21
3 Arquitectura multiagente propuesta	23
3.1 Formulación técnica del sistema	23
3.2 Visión general del sistema	23
3.3 Flujo operativo	23
3.4 Definición del entorno	24
3.4.1 Estado y observaciones	24
3.4.2 Espacio de acciones	25
3.4.3 Transición del entorno	25
3.5 Agentes del sistema	25

3.5.1	Scout	26
3.5.2	Trader	26
3.5.3	Portfolio	26
3.6	Función de recompensa	26
3.6.1	Recompensa individual	27
3.6.2	Recompensa cooperativa	27
3.7	Restricciones de riesgo	27
4	Configuración del entorno y datos	28
4.1	Fuentes de datos	28
4.1.1	Mercado de Steam	28
4.1.2	BUFF y mercados externos	28
4.1.3	OCR e importación manual	29
4.2	Adquisición y persistencia	29
4.2.1	Pipeline de adquisición	30
4.2.2	Modelo de persistencia	30
4.3	Preparación de características	30
4.3.1	Variables de mercado	30
4.3.2	Dataset temporal de trading	31
4.3.3	Variables de cartera	32
4.4	Reglas económicas migradas	32
4.5	Simulador de cartera	32
4.5.1	Operaciones permitidas	32
4.5.2	Comisiones y valoración	32
5	Implementación y operación	34
5.1	Organización del sistema	34
5.2	Flujo operativo de adquisición	34
5.2.1	Sesiones, scraping y trazabilidad	34
5.2.2	Persistencia incremental	35
5.3	Consola de consulta	35
5.4	Reproducibilidad y configuración	36
6	Experimentación y análisis	37
6.1	Diseño experimental	37
6.1.1	Escenarios de evaluación	37
6.1.2	Estrategias de comparación	38
6.2	Validación técnica y pruebas	38
6.3	Métricas	38
6.3.1	Métricas de rentabilidad	39
6.3.2	Métricas de riesgo	39
6.4	Resultados	39
6.4.1	Migración de reglas económicas	39
6.4.2	Resultados del modelo supervisado	40
6.4.3	Protocolo temporal reproducible	40
6.4.4	Ablación de histórico y aumentación	41
6.4.5	Evidencia de pruebas automatizadas	43
6.4.6	Dataset de trading operativo	43
6.4.7	Dificultades y pivotaciones	43

6.4.8	Resultados de la arquitectura multiagente	44
6.5	Experimentos finales pendientes	45
6.6	Discusión	45
6.6.1	Casos favorables	45
6.6.2	Limitaciones observadas	45
6.6.3	Lectura de los resultados	45
7	Conclusiones	46
7.1	Conclusiones principales	46
7.2	Cumplimiento de objetivos	46
7.3	Limitaciones	47
7.4	Trabajo futuro	47
	Bibliografía	48
A	Anexos	49
A.1	Estructura del repositorio	49
A.2	Configuración del entorno	49
A.3	Comandos de datasets y modelos	50
A.4	Parámetros de entrenamiento	50
A.5	Modelo de datos	50
A.6	Resultados adicionales	51

Índice de figuras

2.1	Resumen visual de la pila tecnológica utilizada en el proyecto.	20
3.1	Capas principales de la arquitectura propuesta.	24
6.1	Exploración inicial de modelos y variables temporales en marzo. Un Brier menor indica mejor calibración.	41
6.2	Efecto del histórico y de la aumentación numérica sobre la prueba de marzo.	42

Índice de tablas

3.1	Responsabilidades de los agentes del sistema.	26
4.1	Controles del pipeline OCR.	30
4.2	Entidades principales para datos y evaluación.	31
4.3	Reglas económicas procedentes del Excel operativo.	32
5.1	Distribución funcional del repositorio.	34
5.2	Vistas principales de la consola local.	36
6.1	Fases experimentales desarrolladas hasta el momento.	37
6.2	Bloques de pruebas incorporados a la validación del proyecto.	39
6.3	Resultados supervisados preliminares sobre el dataset reciente.	40
6.4	Exploración inicial de familias en el corte temporal de marzo.	41
6.5	Ablación logística en el corte temporal de marzo.	42
6.6	Ejecución de pruebas automatizadas recuperada.	43
6.7	Pruebas iniciales de las políticas MARL.	44
A.1	Parámetros a versionar en los experimentos.	50

Lista de algoritmos

1	Ciclo general de toma de decisiones	24
2	Decisión cooperativa	25
3	Extracción OCR validada	29
4	Dataset temporal	31
5	Trabajo de scraping trazable	35
6	Ejecución reproducible de un experimento	36
7	Entrenamiento PPO	38

Lista de acrónimos

API Interfaz de programación de aplicaciones.

BUFF Mercado externo de compraventa de activos digitales usado como fuente de precios.

CLI Interfaz de línea de comandos, del inglés Command-Line Interface.

CNY Yuan chino, código internacional de divisa.

CS2 Counter-Strike 2.

CSV Valores separados por comas, del inglés Comma-Separated Values.

CTDE Entrenamiento centralizado y ejecución descentralizada, del inglés Centralized Training with Decentralized Execution.

EUR Euro, código internacional de divisa.

F1 Métrica F1, media armónica entre precisión y exhaustividad.

IA Inteligencia artificial.

MADDPG Gradiente de política determinista profundo multiagente, del inglés Multi-Agent Deep Deterministic Policy Gradient.

MAPPO Optimización proximal de políticas multiagente, del inglés Multi-Agent Proximal Policy Optimization.

MARL Aprendizaje por refuerzo multiagente, del inglés Multi-Agent Reinforcement Learning.

MDP Proceso de decisión de Markov, del inglés Markov Decision Process.

OCR Reconocimiento óptico de caracteres, del inglés Optical Character Recognition.

PPO Proximal Policy Optimization.

PR-AUC Área bajo la curva precisión-exhaustividad, del inglés Precision-Recall Area Under the Curve.

RL Aprendizaje por refuerzo, del inglés Reinforcement Learning.

RLlib Biblioteca de aprendizaje por refuerzo de Ray.

ROC-AUC Área bajo la curva ROC, del inglés Receiver Operating Characteristic Area Under the Curve.

RSI Índice de fuerza relativa, del inglés Relative Strength Index.

TFM Trabajo Fin de Máster.

Introducción

1.1 Motivación

Los mercados de activos digitales asociados a videojuegos han dejado de ser un fenómeno marginal. En el caso de *Counter-Strike 2*, los objetos cosméticos como skins, cuchillos, guantes, pegatinas o cajas se compran y venden de forma continua en diferentes plataformas. Aunque estos activos no tienen una forma física, su precio se comporta como una señal económica observable: cambia con la demanda, rareza, liquidez, condición del objeto, disponibilidad en cada mercado y expectativas de reventa. La literatura sobre bienes virtuales ya señala que el valor de estos objetos depende de atributos funcionales, estéticos y sociales [1].

El interés de este TFM nace de una contradicción bastante clara. Por un lado, existe un mercado activo, con histórico de precios, diferencias entre plataformas y oportunidades de arbitraje. Por otro, buena parte de la gestión sigue dependiendo de hojas de cálculo, comprobaciones manuales y reglas estáticas. Esta forma de trabajo puede servir para un número pequeño de artículos, pero se vuelve frágil cuando se intenta seguir muchos objetos a la vez y reaccionar con rapidez ante cambios de precio.

El caso de *Counter-Strike 2* añade además fricciones propias que hacen que una oportunidad aparente no siempre sea una buena decisión. No basta con comprar un artículo barato: la operación debe evaluarse después de comisiones, conversión de divisa, liquidez disponible, *trade hold* (bloqueo temporal que impide vender o intercambiar un artículo recién adquirido), y coste de oportunidad del capital inmovilizado. En la práctica, una operación rentable en bruto puede dejar de serlo cuando se calcula el beneficio neto, del mismo modo que los entornos de trading automatizado deben modelar costes de transacción y restricciones de liquidez [2].

Antes de este proyecto, parte de estas decisiones ya se apoyaban en un Excel operativo con fórmulas de conversión de moneda, comisiones, beneficio, porcentaje de retorno, fechas de desbloqueo y capital en espera. Ese Excel ha servido como especificación inicial del dominio. No se ha copiado como estructura final, pero sí ha permitido identificar las reglas económicas que debían convertirse en funciones reutilizables dentro del sistema.

La motivación técnica del trabajo es transformar ese proceso manual en una base reproducible de adquisición, simulación y evaluación. El objetivo no es ejecutar compras reales durante el desarrollo del TFM, sino construir un entorno controlado en el que las decisiones puedan estudiarse con datos históricos y reglas de mercado realistas.

1.2 Planteamiento del problema

Abordamos el problema de transformar observaciones dispersas del mercado en decisiones simuladas y justificables. El sistema parte con una lista de candidatos, precios,

liquidez, reglas de comisión y un estado de cartera. Con esa información debe estimar si una oportunidad merece seguimiento, si conviene simular una compra o si debe descartarse por riesgo, falta de datos o beneficio insuficiente. Esta formulación conecta la predicción de señales con la gestión de cartera, donde rentabilidad y riesgo deben analizarse conjuntamente [3].

La dificultad principal está en convertir señales parciales en una decisión trazable. El sistema no debe limitarse a decir si un precio parece subir o bajar, sino justificar por qué una oportunidad pasa a revisión, queda en observación o se descarta. Esa justificación debe apoyarse en datos de entrada, hipótesis económicas y estado de cartera, de forma que cada recomendación pueda auditarse después.

Esta formulación obliga a separar dos niveles. El primero es el nivel de observación y predicción: recopilar datos, limpiarlos, construir características y estimar señales útiles. El segundo es el nivel de decisión: usar esas señales dentro de un simulador de cartera que permita comparar acciones sin arriesgar capital. La arquitectura multiagente se introduce en este segundo nivel como hipótesis de trabajo para repartir responsabilidades entre detección de oportunidades, decisión operativa y control de riesgo.

1.3 Objetivos

El objetivo principal de este TFM es diseñar y evaluar una arquitectura multiagente para apoyar la toma de decisiones en mercados de activos digitales, usando como caso de estudio la economía virtual de *Counter-Strike 2*.

Los objetivos específicos se agrupan en cuatro bloques:

- **Comprensión del dominio:** analizar el mercado de activos digitales de *Counter-Strike 2*, sus plataformas, restricciones, comisiones, divisas, liquidez y efectos del *trade hold*.
- **Construcción de datos:** diseñar un flujo de adquisición y preparación de datos procedentes de Steam, BUFF, SteamDT, OCR y fuentes manuales, manteniendo trazabilidad de fuente, fecha, moneda y calidad del dato.
- **Simulación y predicción:** migrar reglas económicas del proceso manual a código comprobable, construir datasets temporales, entrenar modelos supervisados y generar señales probabilísticas que puedan usarse dentro del entorno de decisión.
- **Decisión multiagente:** formular el problema como un entorno de aprendizaje por refuerzo multiagente, definir agentes especializados, establecer recompensas coherentes con la cartera y evaluar el comportamiento mediante métricas de rentabilidad, riesgo, exposición y capital inmovilizado.

1.4 Alcance y restricciones

El alcance del proyecto se centra en la simulación y en la evaluación. Cualquier decisión generada por el sistema se registra como decisión simulada. Esta restricción es importante porque permite experimentar con estrategias sin introducir riesgos económicos ni depender de una integración operativa completa con plataformas externas.

La arquitectura multiagente se plantea como una forma de separar responsabilidades dentro del proceso de decisión. La adquisición de datos, la predicción supervisada, el simulador económico, los agentes MARL y la evaluación experimental se mantienen como capas diferenciadas para que cada parte pueda validarse, sustituirse o compararse sin comprometer el conjunto del sistema.

Otra restricción relevante es la disponibilidad de histórico alineado entre plataformas. El mercado de Steam y el de BUFF no siempre ofrecen datos con la misma granularidad, moneda, horizonte o estabilidad de acceso. Por ello, una parte del trabajo consiste en construir una base experimental, donde los resultados preliminares se distingan claramente de una validación definitiva.

La metodología distingue entre el aprendizaje de patrones temporales y la evaluación operativa de oportunidades concretas. Esta separación permite aprovechar históricos más estables para entrenar modelos y, al mismo tiempo, incorporar precios puntuales de mercado cuando se simulan decisiones de cartera.

1.5 Metodología de desarrollo

El desarrollo se ha organizado siguiendo el propio avance del proyecto. Primero, se ha estudiado el proceso manual y se han extraído las reglas económicas del Excel operativo. Después, se ha construido una base de adquisición y persistencia para recoger datos de mercado. A continuación, se han preparado conjuntos de datos supervisados y se han evaluado modelos tabulares para obtener señales probabilísticas. Finalmente, se ha implementado un simulador de cartera y un entorno multiagente compatible con PettingZoo y RLlib para realizar pruebas funcionales.

Esta metodología incremental evita entrenar agentes sobre un entorno poco fiable. En lugar de empezar directamente por MARL, el proyecto construye primero las piezas que hacen verificable la decisión: datos, reglas económicas, simulación, métricas y trazabilidad. La arquitectura multiagente se apoya en estas capas, no las sustituye.

1.6 Estructura del documento

La memoria se estructura de forma secuencial siguiendo las fases principales del proyecto. A continuación, se detalla el contenido de cada capítulo:

- **Capítulo 1:** introduce el contexto del proyecto, motivación, planteamiento del problema, objetivos, alcance y metodología de desarrollo seguida.
- **Capítulo 2:** presenta el marco teórico necesario para entender el trabajo. Se tratan mercados de activos digitales, fricciones de mercado, RL, MARL, toma de decisiones en carteras, trabajos relacionados y tecnologías utilizadas.
- **Capítulo 3:** describe la arquitectura multiagente propuesta. Se presenta la formulación técnica del sistema, flujo general, entorno de decisión, agentes, función de recompensa y restricciones de riesgo.
- **Capítulo 4:** expone el entorno de datos y simulación. Se detallan fuentes de adquisición, persistencia, características utilizadas, simulador de cartera, OCR e

interfaz de consulta.

- **Capítulo 5:** describe la implementación y operación del sistema. Presenta la organización del repositorio, el scraping, la persistencia, la consola local y la configuración reproducible.
- **Capítulo 6:** recoge la experimentación realizada. Incluye migración de reglas económicas, construcción de datasets supervisados, evaluación de modelos tabulares, pruebas técnicas y de rendimiento, dataset de trading, estado del entrenamiento MARL, dificultades y pivotaciones detectadas.
- **Capítulo 7:** presenta conclusiones principales, revisa el cumplimiento de objetivos y plantea líneas de trabajo necesarias para completar la validación experimental.
- **Anexos:** reúnen información complementaria sobre estructura del repositorio, configuración del entorno, comandos de ejecución, parámetros de entrenamiento y modelo de datos.

Marco teórico y estado del arte

2.1 Mercados de activos digitales

Un activo digital es un bien que existe dentro de un sistema informático y cuyo valor depende de reglas de acceso, escasez, utilidad percibida y capacidad de intercambio. En videojuegos, estos activos pueden tener una utilidad funcional, estética o social. Lehdonvirta estudia precisamente cómo los atributos funcionales, hedónicos y sociales influyen en la demanda de bienes virtuales [1]. Esta idea encaja bien con el mercado de *Counter-Strike 2*, donde dos objetos técnicamente similares pueden tener precios muy distintos por rareza, apariencia, estado, popularidad o valor simbólico dentro de la comunidad.

Las economías virtuales no son idénticas a los mercados financieros tradicionales, pero comparten algunos elementos: oferta limitada, negociación entre compradores y vendedores, formación de precios, costes de transacción y riesgo de liquidez. La diferencia principal es que el mercado está condicionado por una plataforma propietaria y por reglas del videojuego. El emisor del activo, las restricciones de intercambio y los cambios de diseño pueden alterar el valor de los objetos sin que exista una autoridad financiera externa que proteja al usuario.

2.1.1 Activos virtuales en Counter-Strike 2

En *Counter-Strike 2*, los activos más relevantes para este trabajo son skins, cuchillos, guantes, pegatinas, cajas y otros objetos cosméticos intercambiables. Su precio depende de factores como rareza, condición visual, popularidad del arma, volumen negociado, float, presencia de StatTrak, patrones especiales y demanda puntual. Estos atributos no siempre aparecen de forma limpia en una API, por lo que el sistema debe combinar scraping, histórico, normalización y revisión de nombres para representar correctamente cada variante.

El mercado no está concentrado en una única fuente. Steam actúa como mercado oficial y ofrece señales de precio e histórico, mientras que plataformas externas como BUFF permiten observar precios, buy orders y oportunidades que no aparecen con el mismo formato en Steam. Esta fragmentación crea diferencias de precio entre plataformas, pero también aumenta la dificultad de decidir: una oportunidad puede existir en una fuente y desaparecer antes de que el activo pueda venderse en otra.

2.1.2 Fricciones de mercado

Las fricciones son costes o restricciones que separan el precio observado del beneficio neto. En este proyecto son especialmente importantes las comisiones de plataforma, la conversión entre EUR y CNY, el *spread*, el volumen disponible, las buy orders, el *trade*

hold y el capital bloqueado. Por este motivo, el sistema no puede limitarse a comparar dos precios brutos. Debe calcular el precio neto de entrada, el precio neto de salida y el tiempo durante el cual la posición no puede cerrarse.

La liquidez es otra fricción central. Un activo puede mostrar un margen alto, pero si apenas tiene volumen o si las buy orders son débiles, la venta puede tardar demasiado o producirse a un precio inferior al esperado. En una cartera simulada, esa espera debe tener coste: el capital queda inmovilizado y no puede utilizarse para otras oportunidades.

2.2 Aprendizaje por refuerzo

El aprendizaje por refuerzo estudia problemas en los que un agente interactúa con un entorno, toma acciones y recibe recompensas. A diferencia del aprendizaje supervisado, donde el modelo aprende a reproducir etiquetas históricas, en RL el modelo aprende una política de actuación a partir de las consecuencias de sus acciones [4]. Esta formulación es útil cuando una decisión actual modifica el estado futuro, como ocurre al abrir una posición que queda bloqueada por *trade hold*.

2.2.1 Procesos de decisión de Markov

Un proceso de decisión de Markov, o MDP, se define mediante el estado, las acciones, la transición, la recompensa y el factor de descuento:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \quad G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

G_t resume la recompensa futura descontada desde el instante t . El estado resume la información disponible, la acción representa lo que el agente decide, la transición describe cómo cambia el entorno y la recompensa mide si la decisión ha sido favorable. En el contexto del TFM, el estado incluye variables de mercado y cartera, las acciones son comprar, vender, observar o mantener, y la recompensa se asocia al valor neto de la cartera simulada.

La hipótesis de Markov no implica que el sistema conozca todo el futuro, sino que el estado utilizado contiene la información necesaria para tomar la mejor decisión disponible en ese momento. Por eso es importante construir observaciones ricas: precio, spread, liquidez, probabilidad supervisada, capital disponible, posiciones abiertas y capital bloqueado.

2.2.2 Política, retorno y exploración

La política es la regla que transforma observaciones en acciones. El retorno mide la recompensa acumulada a lo largo del episodio. En un mercado con fricciones, maximizar la recompensa inmediata puede ser mala estrategia: una compra aparentemente rentable puede bloquear capital, aumentar exposición o no encontrar salida. Por ello, la recompensa debe mirar más allá del margen instantáneo y considerar el resultado de cartera.

La exploración permite probar acciones que todavía no se conocen bien. En mercados financieros o cuasi financieros, esta idea debe aplicarse con cuidado. En el TFM, la

exploración se realiza en simulación, no con capital. Esta separación permite probar políticas y recompensas sin trasladar errores del entrenamiento al mercado.

2.3 Aprendizaje por refuerzo multiagente

El aprendizaje por refuerzo multiagente extiende la formulación anterior a varios agentes que interactúan dentro de un mismo entorno. Esta ampliación introduce dificultades nuevas: cada agente aprende mientras los demás también cambian, por lo que el entorno puede volverse no estacionario desde el punto de vista individual. Lowe et al. señalan este problema y proponen entrenar agentes con información adicional durante el entrenamiento para mejorar la coordinación mediante MADDPG [5].

En este TFM, la motivación para usar varios agentes no es decorativa. La decisión de cartera mezcla tareas de naturaleza distinta: detectar oportunidades, ejecutar acciones concretas y controlar exposición global. Separar estas responsabilidades permite que cada agente trabaje con un punto de vista más acotado, siempre que la recompensa cooperativa mantenga alineado el objetivo común.

2.3.1 Cooperación y coordinación entre agentes

La cooperación aparece cuando los agentes comparten un objetivo global. Scout puede detectar oportunidades prometedoras, Trader puede decidir la acción operativa y Portfolio puede limitar decisiones que deterioren la cartera. Ninguno de estos roles tiene sentido de forma aislada si el resultado final no mejora el valor neto ajustado por riesgo.

La coordinación es necesaria porque una decisión localmente buena puede ser globalmente mala. Por ejemplo, comprar un activo con spread atractivo puede ser razonable para Trader, pero no para Portfolio si ya existe demasiada exposición a ese tipo de objeto o si el capital bloqueado supera un límite aceptable. Esta tensión justifica usar recompensas individuales auxiliares y una recompensa cooperativa vinculada a la cartera.

Para N agentes, cada paso combina observaciones y acciones locales. En una tarea cooperativa con recompensa compartida se puede expresar como:

$$\mathbf{o}_t = (o_t^1, \dots, o_t^N), \quad \mathbf{a}_t = (a_t^1, \dots, a_t^N), \quad r_t^1 = \dots = r_t^N = R_t.$$

Cada política recibe solo su observación o_t^i . El estado global se registra para diagnóstico, pero no alimenta todavía un crítico centralizado, por lo que la implementación actual no se presenta como MAPPO completo.

2.3.2 Entrenamiento centralizado y ejecución descentralizada

El paradigma de entrenamiento centralizado y ejecución descentralizada, conocido como CTDE, permite entrenar con más información de la que cada agente tendrá en ejecución. La idea es que durante el entrenamiento puede existir un crítico o una función de valor con acceso al estado global, mientras que en ejecución cada actor decide con su observación local. Este enfoque aparece en trabajos multiagente como MADDPG [5] y en variantes cooperativas basadas en PPO como MAPPO [6].

La elección de PPO como base práctica se debe a su estabilidad y a su disponibilidad en herramientas maduras. Schulman et al. introducen PPO como un método de gradiente de política que limita actualizaciones demasiado agresivas [7]. Para una acción observada, el cociente entre la política nueva y la anterior es:

$$r_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)}, \quad L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right].$$

El recorte evita que una actualización cambie en exceso la probabilidad de una acción. Posteriormente, Yu et al. muestran que PPO puede funcionar como baseline fuerte en juegos cooperativos multiagente cuando se implementa con cuidado [6]. En el proyecto, esta línea se traduce en un entorno PettingZoo y un wrapper para RLlib, pero todavía no se presenta como validación final de MAPPO.

2.4 Toma de decisiones en carteras

La gestión de una cartera no consiste solo en elegir activos con rentabilidad esperada positiva. También debe controlar riesgo, concentración, liquidez, costes de transacción y horizonte temporal. La teoría moderna de carteras introdujo la idea de estudiar rentabilidad y riesgo conjuntamente, en lugar de analizar cada inversión de forma aislada [3]. Aunque el mercado de CS2 no sea un mercado bursátil regulado, la intuición sigue siendo útil: una cartera puede empeorar aunque una operación individual parezca atractiva.

2.4.1 Rentabilidad y riesgo

En este TFM, la rentabilidad se mide mediante beneficio neto, retorno porcentual, saldo final y evolución del valor de cartera. El riesgo se observa mediante drawdown, capital bloqueado, exposición por activo o plataforma y operaciones rechazadas por restricciones. Estas métricas son más informativas que una simple precisión de predicción, porque conectan el modelo con el resultado económico simulado.

El aprendizaje supervisado puede estimar una probabilidad útil, pero esa probabilidad no es una decisión. Una señal de subida o de spread rentable debe pasar por el simulador, por las comisiones, por la liquidez y por el estado de cartera. Esta separación evita que el sistema confunda predecir con invertir.

2.4.2 Liquidez y costes de transacción

Los costes de transacción son centrales en cualquier sistema de trading. En un mercado de baja o media liquidez, pequeñas diferencias de precio pueden desaparecer al aplicar comisiones, conversión de moneda o retrasos de venta. FinRL incorpora explícitamente restricciones como costes de transacción, liquidez y aversión al riesgo en entornos de trading con aprendizaje por refuerzo [2]. Esa orientación resulta útil para este TFM, aunque el dominio sea distinto: el objetivo no es copiar un framework financiero, sino adoptar la disciplina de simular fricciones reales.

En el caso CS2, las fricciones se vuelven todavía más visibles por el *trade hold*. Durante varios días, una posición comprada no puede venderse, aunque el precio cambie.

Esta restricción convierte una oportunidad puntual en una decisión temporal: el sistema debe estimar no solo el margen de entrada, sino la probabilidad de que el margen siga siendo válido cuando la posición pueda cerrarse.

2.5 Aplicaciones relacionadas

Los trabajos de RL financiero muestran que entrenar agentes de trading exige reproducibilidad, backtesting y control de fricciones [2]. Los trabajos de MARL muestran que la coordinación entre agentes puede ser útil, pero también introduce inestabilidad y necesidad de evaluación cuidadosa [5, 6]. El proyecto se sitúa en la intersección de ambas ideas: usa un mercado de activos digitales como entorno de decisión, incorpora señales supervisadas y plantea una arquitectura multiagente cooperativa.

La diferencia principal respecto a una aplicación financiera clásica es el dominio. Los activos de CS2 son bienes virtuales controlados por plataformas, con liquidez fragmentada y reglas propias. La diferencia respecto a una simple herramienta de scraping es el objetivo: no basta con mostrar precios, sino que se busca evaluar decisiones de cartera bajo restricciones. Por ello, el trabajo combina adquisición de datos, simulación económica, predicción supervisada y aprendizaje por refuerzo multiagente dentro de una misma metodología.

2.6 Tecnologías utilizadas

Siguiendo la estructura observada en el TFG de referencia, las tecnologías del proyecto se presentan indicando su función general y su aplicación concreta dentro del sistema. No se enumeran como una lista aislada de herramientas, sino como parte de la justificación técnica de la solución: cada tecnología cubre una necesidad de adquisición, persistencia, predicción, simulación, entrenamiento o validación.

Bloque	Tecnologías principales
Adquisición	Python, Playwright, HTTPX, Steam, SteamDT, BUFF y OCR.
Persistencia	Postgres/Supabase, SQLAlchemy, AsyncPG y PyArrow.
Predicción	Pandas, NumPy, scikit-learn y Joblib.
Simulación	Módulos propios de cartera, riesgo, comisiones y <i>trade hold</i> .
Multiagente	PettingZoo, Ray/RLlib, Gymnasium, PyTorch y PPO.
Validación	Pytest, pytest-asyncio, Ruff, Mypy y pruebas de integración.
Interfaz	HTML, CSS, JavaScript y servidor web local en Python.

Figura 2.1: Resumen visual de la pila tecnológica utilizada en el proyecto.

2.6.1 Herramientas de desarrollo

El proyecto se desarrolla como un monorepo en Python 3.11. Git se utiliza para versionar cambios y mantener trazabilidad de las decisiones técnicas. La estructura separa aplicaciones ejecutables, paquetes reutilizables, documentación, datos derivados

y pruebas, lo que permite evolucionar el sistema sin mezclar adquisición, predicción, simulación y evaluación.

Pytest y pytest-asyncio se emplean para automatizar pruebas unitarias y asíncronas. Ruff se usa como herramienta de análisis estático y estilo, mientras que Mypy permite detectar errores de tipado en módulos críticos. Estas herramientas no forman parte de la lógica de negocio, pero sí de la evidencia de calidad del desarrollo.

2.6.2 Adquisición y tratamiento de datos

La adquisición de datos se implementa principalmente en Python. HTTPX permite consultar fuentes HTTP cuando existe una interfaz estable, mientras que Playwright se reserva para escenarios en los que el dato solo está disponible mediante navegación o renderizado de página. En el caso del OCR, OpenCV y Tesseract permiten extraer información visual cuando el dato no se recibe en formato estructurado.

Pandas, NumPy y PyArrow se utilizan para preparar conjuntos de datos, generar características temporales y conservar datasets derivados en formatos reproducibles. Esta capa conecta las fuentes de mercado con los modelos supervisados y con los episodios del simulador.

2.6.3 Persistencia y modelo de datos

Postgres, a través de Supabase en el planteamiento operativo, actúa como base de persistencia para artículos, observaciones de mercado, históricos, predicciones y decisiones simuladas. AsyncPG y SQLAlchemy facilitan el acceso desde Python y permiten separar consultas, repositorios y lógica de dominio.

La elección de una base relacional se ajusta al tipo de información del proyecto: series temporales, activos, plataformas, precios, señales y decisiones deben poder relacionarse y auditarse posteriormente. Esta trazabilidad es importante para explicar por qué una oportunidad ha sido recomendada, observada o descartada.

2.6.4 Predicción, simulación y entrenamiento multiagente

scikit-learn se utiliza para los modelos supervisados tabulares, especialmente regresión logística, random forest e histogram gradient boosting. Joblib permite persistir modelos entrenados para usarlos después en inferencia. Estas tecnologías cubren la parte predictiva, pero la decisión final se evalúa dentro del simulador de cartera.

La simulación económica se implementa con módulos propios porque las reglas del dominio son específicas: comisiones, conversión de moneda, beneficio neto, *trade hold*, capital bloqueado y restricciones de exposición. Sobre esta base se construye el entorno multiagente con PettingZoo y el adaptador de entrenamiento con Ray/RLlib. PyTorch y Gymnasium forman parte del ecosistema de entrenamiento usado por RLlib.

2.6.5 Interfaz y operación local

La interfaz web local se implementa con HTML, CSS y JavaScript, servida desde Python. Su objetivo es mostrar estado de scraping, datos disponibles, recomendaciones, métricas y tareas operativas. No se presenta como producto final, sino como herramienta

de observabilidad del sistema durante el desarrollo y la evaluación.

Arquitectura multiagente propuesta

3.1 Formulación técnica del sistema

A partir del planteamiento descrito en el Capítulo 1, la propuesta traduce el caso de uso a una tarea de decisión secuencial sobre una cartera simulada de activos digitales, siguiendo la formulación general de los problemas de aprendizaje por refuerzo [4]. En cada instante, el sistema recibe observaciones de mercado, estado de cartera y señales derivadas. A partir de esa información decide si conviene comprar, vender, mantener en observación o rechazar una oportunidad.

La formulación técnica incorpora las fricciones que condicionan cada acción: beneficio neto después de comisiones, cambio de moneda, liquidez disponible, capital ya bloqueado, tiempo de desbloqueo por *trade hold* y exposición acumulada a un mismo activo o plataforma. Con estos elementos, el objetivo del sistema es maximizar el valor total de la cartera simulada bajo restricciones de riesgo y fricción de mercado.

La arquitectura propuesta separa esta formulación en dos niveles. El primer nivel observa y transforma el mercado: extrae datos, los normaliza, los guarda y calcula señales supervisadas. El segundo nivel toma decisiones dentro de un simulador: recibe observaciones ya preparadas, emite acciones y recibe recompensas según el comportamiento de la cartera. Esta separación permite aislar la adquisición de datos, la limpieza, la predicción y el control de riesgo como responsabilidades diferenciadas.

3.2 Visión general del sistema

La arquitectura se organiza en cinco capas: adquisición, persistencia, predicción, decisión multiagente y simulación. Cada capa tiene una responsabilidad concreta y se comunica con la siguiente mediante datos normalizados. Esta decisión de diseño reduce el acoplamiento y permite validar cada parte por separado, una práctica habitual en entornos de trading reproducibles y backtesting [2].

La decisión final siempre queda registrada como simulada. Esto permite evaluar el sistema con métricas de cartera sin exponer capital durante la investigación. La arquitectura puede generar recomendaciones, episodios y métricas, pero no envía órdenes a Steam, BUFF ni ninguna otra plataforma.

3.3 Flujo operativo

El flujo operativo comienza con una lista de candidatos o una búsqueda de mercado. Los agentes extractores consultan las fuentes disponibles, capturan precios, buy orders, volumen y metadatos, y construyen observaciones normalizadas. Después, la persisten-

1. Adquisición	Steam, BUFF, SteamDT, OCR e importaciones manuales capturan observaciones.
2. Persistencia	Postgres/Supabase guarda artículos, históricos, snapshots, predicciones y decisiones simuladas.
3. Predicción	El modelo supervisado genera probabilidades calibradas y señales de mercado.
4. Decisión MARL	Scout, Trader y Portfolio deciden dentro del entorno PettingZoo/RLlib.
5. Simulación	La cartera aplica comisiones, divisas, <i>trade hold</i> , riesgo y métricas.

Figura 3.1: Capas principales de la arquitectura propuesta.

cia conserva el histórico en formato auditable para que el dataset pueda reconstruirse sin depender de una sesión concreta de scraping.

Sobre ese histórico se construyen características de mercado. El modelo supervisado produce una probabilidad calibrada de oportunidad rentable en un horizonte determinado. Esa probabilidad no se interpreta como una orden de compra, sino como una característica adicional que los agentes MARL pueden utilizar. Finalmente, el entorno multiagente genera una observación para cada política, recoge sus acciones y aplica las reglas del simulador.

Algoritmo 1 Ciclo general de toma de decisiones

- 1: Recoger observaciones recientes del mercado.
 - 2: Normalizar precios, moneda, liquidez y metadatos de plataforma.
 - 3: Actualizar el estado de cartera y las variables de riesgo.
 - 4: Calcular señales supervisadas disponibles para el episodio.
 - 5: Generar una observación local para Scout, Trader y Portfolio.
 - 6: Obtener acciones de los agentes y aplicar restricciones de riesgo.
 - 7: Simular la decisión final sobre la cartera.
 - 8: Calcular recompensa, métricas y trazabilidad del episodio.
-

3.4 Definición del entorno

El entorno representa una cartera simulada que evoluciona a partir de datos históricos. Cada episodio contiene una secuencia de observaciones de mercado y un estado interno formado por saldo disponible, posiciones abiertas, posiciones bloqueadas, fechas de desbloqueo y métricas de exposición.

3.4.1 Estado y observaciones

Las observaciones combinan variables de mercado y variables de cartera. Entre las variables de mercado se incluyen precios de Steam y BUFF normalizados a EUR, precios originales en su moneda nativa, volumen, liquidez, spread bruto, spread neto, buy orders, edad de la variante y señales temporales como retornos, medias móviles, RSI o desviaciones de ventas. Entre las variables de cartera se incluyen efectivo disponible,

capital bloqueado, posiciones abiertas, exposición por activo y estado de desbloqueo.

La salida del modelo supervisado se incorpora como una probabilidad calibrada de que una oportunidad mantenga un spread rentable en el horizonte definido. Esta señal permite aprovechar aprendizaje supervisado sin convertirlo en una decisión rígida. Si la probabilidad es alta pero la cartera está sobreexpuesta o el capital está bloqueado, el entorno puede rechazar o posponer la operación.

3.4.2 Espacio de acciones

El espacio de acciones se mantiene deliberadamente simple. Las acciones principales son comprar, vender, mantener y observar. En la parte de compra o venta puede añadirse un tamaño de posición discreto para limitar el número de combinaciones. Esta simplificación permite entrenar y depurar el entorno antes de introducir acciones continuas o reglas de asignación más complejas.

En la versión implementada, Scout puede ignorar o marcar una oportunidad, Trader puede mantener o solicitar la compra de una unidad y Portfolio puede rechazar o aprobar el riesgo. La compra solo se ejecuta cuando los tres agentes eligen la acción favorable y el candidato cumple los límites.

Algoritmo 2 Decisión cooperativa

Entrada: Candidato c_t , estado de cartera s_t y observaciones locales

Salida: Transición de cartera y recompensa compartida R_t

```

1:  $a_t^S \leftarrow \pi_S(o_t^S)$  ▷ Scout marca o ignora
2:  $a_t^T \leftarrow \pi_T(o_t^T, m_t^T)$  ▷ Trader mantiene o compra
3:  $a_t^P \leftarrow \pi_P(o_t^P, m_t^P)$  ▷ Portfolio rechaza o aprueba
4: if  $a_t^S = \text{marcar}$  and  $a_t^T = \text{comprar}$  and  $a_t^P = \text{aprobar}$  and  $c_t$  cumple los límites
   then
5:   Abrir una posición simulada y bloquear su capital
6: else
7:   Mantener el saldo y registrar el motivo de no ejecución
8: end if
9: Actualizar posiciones liquidables, saldo, exposición y drawdown
10: Calcular  $R_t$  y devolver la misma recompensa a los tres agentes
```

3.4.3 Transición del entorno

Cuando se ejecuta una compra simulada, el saldo disponible disminuye y se abre una posición. Esa posición queda bloqueada durante el periodo de *trade hold*. Mientras está bloqueada no puede venderse, pero sí afecta a la exposición y al capital inmovilizado. Cuando se ejecuta una venta simulada, el simulador calcula el ingreso neto aplicando comisiones, conversión de moneda y reglas de valoración. El estado de cartera se actualiza después de cada paso.

3.5 Agentes del sistema

La arquitectura distingue entre agentes extractores y agentes MARL. Los extractores no se entrenan mediante aprendizaje por refuerzo y no deciden operaciones. Su trabajo

es adquirir datos desde Steam, BUFF, SteamDT, OCR o CSV/API y transformarlos en observaciones persistidas. Los agentes MARL, en cambio, operan dentro del entorno de simulación.

Tipo	Agente	Responsabilidad
Extractor	Steam/BUFF	Captura precios, históricos, buy orders, volumen y errores de plataforma.
Extractor	SteamDT	Descubre candidatos y ayuda a priorizar artículos que merecen scraping profundo.
Extractor	OCR/CSV	Convierte capturas o ficheros manuales en observaciones estructuradas.
MARL	Scout	Detecta oportunidades a partir de spread, liquidez y probabilidad supervisada.
MARL	Trader	Decide comprar, vender, mantener u observar una oportunidad concreta.
MARL	Portfolio	Controla capital, exposición, <i>trade hold</i> , drawdown y restricciones globales.

Tabla 3.1: Responsabilidades de los agentes del sistema.

3.5.1 Scout

Scout se encarga de detectar oportunidades. Su observación está orientada a señales de precio, spread, liquidez y probabilidad supervisada. Su papel no es decidir toda la operación, sino marcar qué activos parecen merecer atención dentro del episodio.

3.5.2 Trader

Trader decide la acción operativa principal: comprar, vender, mantener u observar. Para ello considera la señal de Scout, el estado de mercado, el posible beneficio neto y el tamaño de posición permitido. Su responsabilidad está más cerca de la ejecución de una oportunidad concreta.

3.5.3 Portfolio

Portfolio controla la coherencia global de la cartera. Su observación incorpora saldo, capital bloqueado, posiciones abiertas, exposición máxima y restricciones de riesgo. Este agente puede penalizar o bloquear decisiones que parezcan rentables de forma aislada pero que deterioren el perfil global de la cartera.

3.6 Función de recompensa

La recompensa debe reflejar el objetivo del sistema: mejorar el valor neto de la cartera sin ignorar riesgo ni fricción. Una recompensa basada solo en comprar barato o en detectar subida de precio sería incompleta, porque no tendría en cuenta si la operación puede cerrarse, si el capital queda bloqueado o si el spread desaparece antes de poder vender.

3.6.1 Recompensa individual

Las señales individuales pueden ayudar a entrenar comportamientos especializados. Scout puede recibir recompensa por identificar oportunidades que más adelante resultan útiles. Trader puede recibirla por ejecutar acciones con beneficio neto y Portfolio por evitar sobreexposición, operaciones ilíquidas o capital bloqueado excesivo. Estas recompensas no sustituyen al objetivo global, pero pueden guiar el aprendizaje de cada rol.

3.6.2 Recompensa cooperativa

La recompensa cooperativa se vincula al valor total de la cartera simulada. Debe considerar beneficio realizado, valoración de posiciones abiertas, capital bloqueado, drawdown, operaciones rechazadas por riesgo y costes de transacción, en línea con el tratamiento conjunto de rentabilidad y riesgo de la teoría de carteras [3]. En la implementación actual, el entorno MARL ya expone métricas como recompensa media, operaciones, efectivo, beneficio realizado, drawdown, capital bloqueado y posiciones abiertas.

3.7 Restricciones de riesgo

Las restricciones de riesgo se introducen antes de escalar el entrenamiento. Las más importantes son: no usar capital inexistente, limitar exposición por activo o plataforma, rechazar operaciones con liquidez insuficiente, respetar el *trade hold*, controlar el capital bloqueado y separar claramente simulación de operación. Estas reglas son sencillas, pero evitan que los agentes aprendan políticas imposibles de ejecutar en el mercado.

La salida del sistema debe ser siempre auditable. Cada predicción, acción o decisión simulada debe poder relacionarse con su fuente de datos, versión de modelo, episodio y configuración de simulación. Esta trazabilidad es necesaria para interpretar resultados y para distinguir entre error de datos, error de modelo y mala política de decisión.

Configuración del entorno y datos

4.1 Fuentes de datos

El sistema trabaja con varias fuentes porque el mercado de CS2 está fragmentado. Ninguna fuente aislada contiene toda la información necesaria para decidir. Steam aporta precio, histórico y señales del mercado principal. BUFF permite comparar oportunidades y buy orders. SteamDT ayuda a descubrir candidatos. El OCR permite capturar datos cuando no existe una API cómoda. Los CSV o importaciones manuales sirven para integrar muestras controladas o material histórico.

4.1.1 Mercado de Steam

Steam actúa como una de las fuentes principales de precio e histórico. En el proyecto se capturan variantes reales de artículos, precios actuales, moneda, buy orders y puntos históricos cuando están disponibles. La información se normaliza para que pueda combinarse con otras plataformas y para evitar que las decisiones dependan de nombres o formatos propios de una fuente.

En la práctica, Steam no se usa solo como una tabla de precios. También aporta señales de liquidez y de comportamiento temporal. El histórico permite construir retornos, medias móviles, volatilidad y desviaciones frente a ventas recientes. Estas variables son necesarias para que el modelo no mire únicamente una fotografía puntual del mercado.

4.1.2 BUFF y mercados externos

BUFF se utiliza como mercado externo de comparación. Sus precios se reciben principalmente en CNY, por lo que el sistema conserva tanto el valor original como su conversión a EUR. Esta doble representación permite auditar el dato original y, al mismo tiempo, entrenar modelos sobre una moneda canónica. Las buy orders de BUFF son especialmente útiles porque ayudan a estimar si una salida tiene demanda o si el precio observado es poco ejecutable.

SteamDT se utiliza como apoyo para descubrir candidatos y estrategias de búsqueda, no como fuente única de decisión. Su función dentro del sistema es reducir el espacio de búsqueda inicial y ayudar a priorizar qué artículos merece la pena observar con más profundidad.

Tras las primeras pruebas, BUFF se separa del entrenamiento principal. El histórico masivo se apoya en Steam, mientras que BUFF se usa como precio vivo de entrada cuando se evalúa un flip concreto hacia Steam. De este modo el modelo no intenta predecir el precio de BUFF ni depende de scraping recurrente de BUFF para aprender patrones temporales.

4.1.3 OCR e importación manual

El OCR se incorpora como vía secundaria de adquisición. Su objetivo es convertir capturas, tablas o interfaces no estructuradas en observaciones equivalentes a las obtenidas por scraping o API. El flujo incluye preprocesado de imagen, reconocimiento de texto, parsing estructurado, validación y persistencia. Aunque es más frágil que una fuente estructurada, resulta útil cuando una plataforma no expone datos cómodamente o cuando se quiere incorporar material histórico procedente de capturas.

La implementación actual se centra en tablas y mensajes de mercado, que son las capturas con una estructura suficientemente regular para poder validarse. OpenCV amplía la imagen por un factor dos, la convierte a escala de grises, mejora el contraste local con CLAHE, reduce ruido y aplica una binarización adaptativa. Tesseract devuelve las palabras y una confianza por palabra. La confianza agregada utilizada por el pipeline es la media de las n palabras reconocidas con confianza válida:

$$c_{\text{OCR}} = \frac{1}{100n} \sum_{j=1}^n c_j.$$

El parser normaliza espacios y separadores decimales, identifica precio, moneda, calidad, plataforma y volumen cuando aparecen en la línea. Solo se convierten en observaciones las extracciones con $c_{\text{OCR}} \geq 0,5$ y con un precio positivo menor de un millón de unidades monetarias. Esta regla evita que una captura dudosa entre silenciosamente al histórico.

Algoritmo 3 Extracción OCR validada

Entrada: Captura I , confianza mínima τ y contexto de fuente

Salida: Observaciones normalizadas o rechazo trazable

```
1: Cargar  $I$  y aplicar escala, contraste, reducción de ruido y binarización
2: Ejecutar Tesseract y calcular  $c_{\text{OCR}}$ 
3: if  $c_{\text{OCR}} < \tau$  then
4:   Rechazar la captura y conservar su referencia para revisión
5: else
6:   for all líneas reconocidas do
7:     Extraer artículo, calidad, plataforma, precio, moneda y volumen
8:     if precio y moneda son válidos then
9:       Construir identificador canónico y observación con la confianza obtenida
10:    end if
11:  end for
12: end if
```

Las importaciones manuales y CSV cumplen una función complementaria. Permiten introducir muestras controladas, fixtures o históricos parciales sin depender de una sesión de navegador. Esta vía ha sido útil para validar parsers, modelos de persistencia y reglas económicas antes de automatizar el flujo completo.

4.2 Adquisición y persistencia

Etapa	Salida	Control aplicado
Preprocesado	Imagen binaria ampliada	CLAHE, reducción de ruido y umbral adaptativo.
Reconocimiento	Texto y confianza por palabra	Media de confianzas válidas de Tesseract.
Parsing	Campos de mercado normalizados	Precio, moneda, calidad, plataforma y volumen.
Validación	Observación o rechazo	Confianza mínima, precio positivo y rango máximo.

Tabla 4.1: Controles del pipeline OCR.

4.2.1 Pipeline de adquisición

La adquisición se organiza como una capa de agentes extractores. Estos procesos pueden ejecutarse como comandos CLI, jobs o servicios programados. Su función es capturar datos, normalizarlos, validar errores y guardarlos con trazabilidad. El flujo incluye scraping, refresco de históricos, OCR, importación manual y un bucle operativo local que puede ejecutar scraping y actualización periódica.

La adquisición periódica contempla cookies o sesiones, delays configurables, límites de concurrencia, reintentos, deduplicación y registro de anomalías. Esta parte es importante porque las fuentes no siempre son estables: pueden cambiar el formato, limitar el ritmo de acceso o devolver información incompleta.

4.2.2 Modelo de persistencia

La persistencia se apoya en Supabase/Postgres. La decisión de diseño es tratar la base de datos como fuente de verdad y mantener el histórico de forma append-only siempre que sea posible. Las tablas operativas principales son `market_items` y `market_history_points`. La primera mantiene una fila por variante de artículo y estado actual por plataforma. La segunda guarda series temporales en formato largo por artículo, plataforma, fecha y métrica.

También se define un esquema canónico más amplio con entidades como `assets`, `platforms`, `market_observations`, `predictions`, `investment_decisions` y `simulated_positions`. Esta separación permite evolucionar desde el esquema operativo actual hacia una representación más trazable y auditable.

4.3 Preparación de características

4.3.1 Variables de mercado

Las características de mercado incluyen precios Steam y BUFF normalizados, precios originales, spread bruto, spread neto, buy orders, volumen, liquidez, número de listings, antigüedad de variante y señales temporales. En la fase supervisada se han explorado variables de momentum y reversión como retornos a 1, 3 y 7 días, distancia a medias móviles, RSI, volatilidad y desviaciones de ventas. Esta combinación de señales tempo-

Entidad	Uso dentro del TFM
<code>market_items</code>	Estado actual de cada variante por plataforma.
<code>market_history_points</code>	Serie temporal en formato largo para precios, buy orders y métricas.
<code>predictions</code>	Salidas versionadas del modelo supervisado.
<code>investment_decisions</code>	Decisiones simuladas y razones asociadas.
<code>simulated_positions</code>	Posiciones abiertas, bloqueadas o cerradas por el simulador.

Tabla 4.2: Entidades principales para datos y evaluación.

rales y restricciones operativas sigue la misma lógica que los entornos de trading con control de fricciones [2].

La exploración inicial detectó que algunas señales temporales aportan información moderada, mientras que ciertas variables categóricas de alta cardinalidad, como identificadores concretos de skins, deben tratarse con cuidado para evitar sobreajuste. Por ello, el entrenamiento separa variables de evaluación y variables de entrenamiento, y permite realizar ablations sin reconstruir todo el dataset.

4.3.2 Dataset temporal de trading

Para cada observación de un artículo en el día t , el constructor busca la primera salida disponible a partir de $t + h$, donde $h = 8$ días representa el bloqueo aplicado por el simulador. La etiqueta no representa una subida de precio aislada, sino una operación que supera los umbrales económicos tras aplicar comisiones y conversión:

$$y_t = 1 \iff \pi_{t+h} \geq \pi_{\min} \wedge \rho_{t+h} \geq \rho_{\min}.$$

Los conjuntos se separan por fecha y se elimina una franja de purga antes de cada frontera para que una salida futura no coincida con el entrenamiento de un periodo anterior.

Algoritmo 4 Dataset temporal

Entrada: Históricos Steam/BUFF, horizonte h , fechas de corte y purga g

Salida: Conjuntos de entrenamiento, validación y prueba trazables

- 1: **for all** observaciones (i, t) ordenadas por artículo y fecha **do**
 - 2: Calcular variables disponibles hasta t
 - 3: Buscar la primera salida del artículo i a partir de $t + h$
 - 4: **if** la salida es válida **then**
 - 5: Calcular π_{t+h} , ρ_{t+h} y la etiqueta y_t
 - 6: Conservar (i, t) , sus variables y la fecha de salida
 - 7: **end if**
 - 8: **end for**
 - 9: Excluir g días antes de cada frontera temporal
 - 10: Separar por fecha y guardar filas, artículos y rango de cada conjunto
-

4.3.3 Variables de cartera

Las variables de cartera proceden del simulador. Incluyen saldo disponible, valor de posiciones abiertas, capital bloqueado por *trade hold*, fecha de desbloqueo, exposición por activo, exposición por plataforma, beneficio realizado y drawdown. Estas variables son necesarias para que el agente Portfolio no decida únicamente por oportunidad local, sino por estado global de la cartera, coherentemente con la evaluación conjunta de rentabilidad y riesgo [3].

4.4 Reglas económicas migradas

El simulador reproduce las reglas económicas que antes estaban dispersas en el Excel operativo. No intenta copiar la hoja de cálculo, sino migrar sus reglas útiles a código comprobable: conversión EUR/CNY, comisiones, beneficio neto, rentabilidad porcentual, fecha de desbloqueo y capital bloqueado.

Regla	Uso en la simulación
Conversión EUR/CNY	Normaliza precios de Steam y BUFF para compararlos en una moneda común.
Fee BUFF 0,975	Estima el ingreso neto al vender en BUFF.
Fee Steam 0,87	Representa la comisión de mercado en ventas Steam.
Cash-out conservador	Permite simular pérdida adicional al convertir saldo Steam en liquidez externa.
<i>Trade hold</i>	Bloquea posiciones durante un horizonte conservador de ocho días.
Beneficio neto	Calcula diferencia entre ingreso neto de venta y coste de entrada.

Tabla 4.3: Reglas económicas procedentes del Excel operativo.

4.5 Simulador de cartera

4.5.1 Operaciones permitidas

Las operaciones básicas son comprar, vender y mantener. Una compra abre una posición con precio de entrada, cantidad, plataforma y fecha de bloqueo. Una venta cierra una posición si está disponible y calcula el beneficio realizado. Mantener no modifica la posición, pero puede afectar al rendimiento si el mercado se mueve o si el capital sigue inmovilizado.

4.5.2 Comisiones y valoración

El Excel operativo utiliza reglas de referencia que se han convertido en hipótesis de simulación. BUFF se modela con un factor neto de venta de 0,975. Steam usa una comisión de mercado del 0,87 y, en escenarios conservadores de salida a banco, puede aplicarse una pérdida adicional del 20 %. La conversión inicial simplificada usa 1 EUR = 8 CNY, aunque queda pendiente versionar tipos de cambio por fecha si se incorporan

fuentes externas.

El *trade hold* se modela de forma conservadora como ocho días desde la compra, equivalente a siete días más una ventana de desbloqueo. Esta simplificación evita declarar vendible una posición demasiado pronto.

Implementación y operación

5.1 Organización del sistema

La implementación se ha organizado como un monorepo para separar el código de dominio, los conectores de adquisición y las herramientas de operación. La separación no responde solo a una preferencia de estructura. Permite ejecutar un scraper, repetir un entrenamiento o consultar el panel sin mezclar esa lógica con las reglas económicas y con los contratos de datos.

Módulo	Responsabilidad
<code>packages/domain y contracts</code>	Identificadores canónicos, tipos de datos y validaciones compartidas.
<code>packages/persistence</code>	Conexión a PostgreSQL y repositorios de artículos, histórico y señales.
<code>packages/simulation</code>	Comisiones, posiciones, bloqueos, riesgo y métricas de cartera.
<code>packages/prediction</code>	Construcción de variables, entrenamiento, calibración e inferencia supervisada.
<code>packages/marl</code>	Entorno paralelo, recompensas, adaptador Petting-Zoo y entrenamiento PPO.
<code>packages/vision</code>	Preprocesado de imágenes, OCR y parsing de observaciones.
<code>apps/cli y apps/web</code>	Comandos reproducibles y consola local de consulta.

Tabla 5.1: Distribución funcional del repositorio.

Las capas se comunican mediante contratos explícitos. Por ejemplo, un extractor no entrega directamente una recomendación. Entrega una observación con artículo, plataforma, fecha, precio, moneda, volumen, origen y calidad. La persistencia conserva esta información y los módulos de datos construyen después las variables necesarias para cada experimento. Esta secuencia facilita localizar un error sin tener que rehacer todo el flujo.

5.2 Flujo operativo de adquisición

5.2.1 Sesiones, scraping y trazabilidad

Las fuentes que necesitan navegación se ejecutan con Playwright en un navegador visible cuando es necesario iniciar sesión. La sesión no se guarda en el repositorio ni

en el archivo de configuración. Se almacena localmente como estado de navegador y se renueva desde la pantalla de sesiones. Esto evita que las credenciales formen parte de los experimentos o de los commits.

Cada trabajo de scraping registra inicio, conectores utilizados, reintentos, progreso, fichero de log y resultado final. Los límites de concurrencia, el tamaño de lote y el tiempo de espera son configurables. El objetivo no es ejecutar el mayor número de peticiones posible, sino mantener un proceso que pueda detenerse, reanudarse y auditarse si una fuente cambia su interfaz.

Algoritmo 5 Trabajo de scraping trazable

Entrada: Lista de candidatos, conectores activos y configuración de trabajo

Salida: Observaciones persistidas y estado de ejecución

```
1: Crear identificador de trabajo y registrar el inicio
2: for all lotes de candidatos do
3:   for all conectores habilitados do
4:     Consultar la fuente con sesión, límite y espera configurados
5:     Normalizar precio, moneda, disponibilidad y fecha de observación
6:     Validar el resultado y registrar anomalías o reintentos
7:   end for
8:   Persistir observaciones válidas y actualizar el progreso
9: end for
10: Registrar resultado, cobertura y referencia del log
```

5.2.2 Persistencia incremental

El modelo operativo almacena el estado actual en `market_items` y las series en `market_history_points`. Guardar el histórico en formato largo permite incorporar nuevas métricas sin alterar el esquema de una tabla ancha. Una observación incluye el origen y la fecha, por lo que una variación de precio puede relacionarse con la fuente y con la ejecución que la recogió.

La deduplicación se realiza antes de persistir y las actualizaciones de estado se comprueban con pruebas de integración. En la validación ejecutada se insertó un snapshot, se actualizó una cotización y se guardó una señal vinculada a un artículo. Cada prueba revierte su transacción al terminar, de modo que las comprobaciones no contaminan el histórico de experimentación.

5.3 Consola de consulta

La consola local convierte el estado técnico en una interfaz de revisión. Su pantalla principal separa la bandeja de oportunidades del detalle de la oportunidad seleccionada. La bandeja concentra el nombre del artículo, la ruta, el margen y los precios Steam y BUFF. El detalle explica qué señal la ha producido, qué datos faltan y qué restricciones de cartera intervienen.

El panel no ejecuta compras. Las acciones de abrir Steam o BUFF sirven para que la persona usuaria revise la oportunidad en su fuente. Los estados *revisar*, *observar* y *bloqueado* representan una decisión explicable, no una orden automática. Esta res-

tricción es importante mientras la validación estadística y multiagente sigue en fase exploratoria.

Vista	Información que hace visible
Oportunidades	Ruta de compra y venta, precios en EUR y CNY, margen estimado y estado de la señal.
Scraper	Trabajo activo, porcentaje, conectores, logs y resultado de la ejecución.
Sesiones	Vigencia de Steam y BUFF y acción para volver a capturar el estado local.
Modelo	Versión, conjunto usado, métricas y carácter experimental de la salida.
Agentes y riesgo	Roles Scout, Trader y Portfolio, capital bloqueado, límites y justificación de rechazos.

Tabla 5.2: Vistas principales de la consola local.

5.4 Reproducibilidad y configuración

La configuración se versiona mediante un fichero de ejemplo sin secretos. Para ejecutar la persistencia basta con definir `DATABASE_URL`. El resto de opciones habilita funciones concretas: ruta de Tesseract, token local del servidor de scraping, límites de concurrencia o parámetros de sesiones. Las claves de una base de datos y los estados de navegador quedan excluidos del control de versiones.

Los comandos de construcción de datasets y entrenamiento guardan el rango temporal, las columnas, los tamaños de cada partición, las tasas de clase, los hiperparámetros y las métricas obtenidas. Esta información permite distinguir dos situaciones que visualmente podrían parecer iguales: ejecutar un modelo con más datos y repetir exactamente una configuración anterior.

Algoritmo 6 Ejecución reproducible de un experimento

Entrada: Dataset versionado, fechas de corte, configuración y semilla

Salida: Informe, figura y modelo asociados a una ejecución

- 1: Construir entrenamiento, validación y prueba respetando el orden temporal
 - 2: Guardar el esquema de variables y las tasas de clase
 - 3: Entrenar los candidatos con la configuración declarada
 - 4: Elegir parámetros usando solo validación
 - 5: Evaluar una vez sobre prueba y guardar métricas
 - 6: Generar tabla o figura desde el informe guardado
-

Este procedimiento se aplica también a las pruebas automatizadas. La suite se ejecuta antes de incorporar una modificación a la memoria, de forma que las tablas y gráficas se apoyen en componentes que han pasado las comprobaciones de dominio, persistencia, OCR, scraping, simulación, MARL y panel web.

Experimentación y análisis

6.1 Diseño experimental

La experimentación se ha desarrollado por fases. Primero, se han migrado reglas económicas y estructuras de datos. Después, se han construido datasets supervisados para estudiar señales de mercado. Más adelante, se ha creado un dataset de trading basado en precios Steam/BUFF y se ha implementado el entorno multiagente. Esta secuencia evita entrenar agentes MARL sobre un entorno que todavía no representa bien el problema económico.

Fase	Objetivo	Estado
1	Migrar reglas económicas del Excel a simulador comprobable.	Implementada y cubierta por pruebas.
2	Construir datasets supervisados y explorar características temporales.	Implementada con resultados preliminares.
3	Comparar modelos supervisados tabulares y umbrales operativos.	Implementada como smoke experimental.
4	Crear dataset operativo de trading Steam/BUFF con fees y horizonte.	Implementada, pero limitada por desbalance.
5	Ejecutar smoke MARL con Petting-Zoo, RLlib/PPO y tres políticas.	Funcional, no validación final.

Tabla 6.1: Fases experimentales desarrolladas hasta el momento.

6.1.1 Escenarios de evaluación

Se han usado dos tipos de datasets. El primero predice dirección de mercado sobre histórico disponible y permite estudiar señales temporales de forma amplia. El segundo intenta modelar oportunidades reales de trading Steam/BUFF con precios normalizados, fees y horizonte temporal. Este segundo dataset es más cercano al objetivo del TFM, pero también es más exigente porque necesita histórico alineado entre plataformas.

El criterio de evaluación no es solo predictivo. Una señal puede tener buena precisión y aun así no ser operativa si aparece pocas veces, si depende de artículos concretos o si no supera costes de transacción. Por eso las métricas del modelo supervisado se interpretan como apoyo a la decisión, no como resultado económico final.

6.1.2 Estrategias de comparación

La comparación experimental se plantea en tres niveles. El primero es un baseline simple o dummy para comprobar que los modelos aprenden algo más que la tasa base. El segundo compara modelos supervisados tabulares como regresión logística, random forest e histogram gradient boosting, familias ampliamente usadas para clasificación tabular [8, 9]. El tercero, todavía pendiente de evaluación completa, comparará la arquitectura MARL frente a baselines de agente único y ablations con y sin probabilidad supervisada.

Algoritmo 7 Entrenamiento PPO

Entrada: Entorno paralelo, políticas Scout, Trader y Portfolio, semillas y pasos

Salida: Checkpoint y métricas de validación y prueba

- 1: Inicializar una política PPO por rol y las máscaras de acción del entorno
 - 2: **for** cada iteración de entrenamiento **do**
 - 3: Ejecutar episodios y registrar $(o_t^i, a_t^i, R_t, m_t^i)$
 - 4: Estimar ventajas $\hat{A}_t$ a partir de las transiciones registradas
 - 5: Optimizar cada política con el objetivo recortado de PPO
 - 6: **end for**
 - 7: Evaluar el checkpoint en un bloque temporal posterior sin actualizar parámetros
 - 8: Guardar saldo, operaciones, acciones inválidas y recompensa por semilla
-

6.2 Validación técnica y pruebas

Las pruebas no se tratan como material auxiliar, sino como parte de la evidencia experimental del proyecto. Antes de interpretar resultados de modelos o agentes, se comprueba que las piezas que producen esos resultados funcionan de forma aislada: reglas económicas, parsing de mercado, persistencia, construcción de datasets, inferencia supervisada, simulador de cartera, restricciones de riesgo, entorno PettingZoo, adaptador RLlib, comandos de episodios y panel web.

La suite automatizada se organiza principalmente con Pytest y se complementa con Ruff y Mypy. Las pruebas unitarias cubren el comportamiento determinista del simulador, los cálculos heredados del Excel, el OCR, SteamDT, Steam, BUFF, los datasets supervisados, el dataset de trading, el servicio de scoring y los componentes MARL. También existe una prueba de integración para repositorios de persistencia, separada porque depende de una base de datos disponible.

Las pruebas de rendimiento se integran en la evaluación del capítulo porque condicionan la utilidad del sistema. En este proyecto el rendimiento no se mide solo por tiempo de ejecución, sino también por cobertura de datos, número de señales generadas, coste de entrenamiento, capital bloqueado, drawdown y operaciones rechazadas. Los smoke tests de entrenamiento y los episodios MARL registran estas métricas para comprobar que el sistema no solo ejecuta, sino que produce salidas interpretables y comparables entre configuraciones.

6.3 Métricas

Bloque	Cobertura principal	Evidencia que aporta
Reglas económicas y cartera	Excel, simulador, riesgo y capital bloqueado.	Verifica fees, conversión, beneficio neto, <i>trade hold</i> , posiciones y rechazos por riesgo.
Datos y scraping	Parsing, SteamDT, Steam, BUFF, OCR y pipeline streaming.	Comprueba que las fuentes se normalizan antes de construir señales o históricos.
Modelos supervisados	Dataset, entrenamiento, inferencia y scoring de oportunidades.	Valida generación de características, carga de modelos e inferencia operativa.
Simulación MARL	Entorno, episodios, recompensas, PettingZoo y adaptador RLlib.	Comprueba acciones, observaciones y compatibilidad con entrenamiento multiagente.
Interfaz y operación	Panel web, servidor de scraping y payload operativo.	Verifica que la interfaz expone estado, recomendaciones y ejecución de tareas.

Tabla 6.2: Bloques de pruebas incorporados a la validación del proyecto.

6.3.1 Métricas de rentabilidad

Las métricas de rentabilidad principales son beneficio neto, retorno porcentual, saldo final y número de señales operativas. En los modelos supervisados se usan además precisión, recall, F1, ROC-AUC, PR-AUC y Brier score. Para el uso operativo se da especial importancia a la precisión en umbrales altos, porque una señal menos frecuente pero más fiable puede ser más útil que muchas señales ruidosas. La calibración de probabilidades es relevante porque la salida del modelo se usa como señal de decisión, no solo como etiqueta binaria [10].

6.3.2 Métricas de riesgo

Las métricas de riesgo incluyen drawdown, capital bloqueado medio, posiciones abiertas, exposición por activo o plataforma, operaciones rechazadas por riesgo e impacto del *trade hold*. Estas métricas son necesarias para evaluar si una estrategia es realmente utilizable y no solo rentable en una métrica aislada.

6.4 Resultados

6.4.1 Migración de reglas económicas

La primera fase ha consistido en convertir reglas del Excel operativo en funciones comprobables. Se han migrado conversiones EUR/CNY, factores de comisión, cálculo de

beneficio neto, rentabilidad porcentual, fecha de desbloqueo y capital bloqueado. Esta fase no produce un resultado de rentabilidad por sí misma, pero es necesaria para que los experimentos posteriores midan el mismo dominio que se utilizaba manualmente.

El resultado principal de esta fase es metodológico: las reglas económicas pasan de estar dispersas en fórmulas de hoja de cálculo a estar encapsuladas en módulos de simulación, con pruebas unitarias y parámetros explícitos. Esto reduce errores silenciosos y permite repetir un experimento con la misma configuración.

6.4.2 Resultados del modelo supervisado

La fase supervisada ha producido varios experimentos preliminares. En el dataset reciente de un año, los modelos random forest han obtenido señales útiles en umbrales altos, aunque con un número limitado de casos. La selección por precisión a umbral 0,8 ha resultado más alineada con el objetivo operativo que seleccionar solo por ROC-AUC.

Experimento	Modelo	Umbral	Resultado en test
Smoke reciente	Random forest d18	0,8	Precisión 0,946 con 92 señales
Selección por precisión	Random forest d10	0,8	Precisión 0,961 con 77 señales
Default operativo	Random forest d18	0,85	Precisión 0,962 con 53 señales

Tabla 6.3: Resultados supervisados preliminares sobre el dataset reciente.

Estos resultados no se consideran definitivos. El número de señales en umbrales altos es reducido y no permite afirmar que el sistema generalice por artículo o por periodo. Sí indican que el pipeline de entrenamiento, calibración, selección por umbral y evaluación temporal funciona.

6.4.3 Protocolo temporal reproducible

La validación se ha repetido el 10 de agosto de 2026 con los datos disponibles desde el 1 de diciembre de 2025. Cada ejemplo representa la posibilidad de comprar un artículo en BUFF y venderlo en Steam tras un horizonte de ocho días. La etiqueta vale uno cuando, después de aplicar las comisiones configuradas, el retorno supera el 10 %. El precio de entrada se toma de BUFF y la salida de Steam se valora con el factor de venta de 0,87 empleado por el simulador.

El orden temporal se conserva en todas las fases. Para el corte de marzo se usaron 171 ejemplos de entrenamiento, 76 de validación y 95 de prueba. El entrenamiento termina el 20 de enero, la validación ocupa del 2 al 20 de febrero y la prueba comprende del 4 al 28 de marzo. Se eliminan ocho días antes de cada frontera para que el horizonte futuro de una observación no alcance el bloque siguiente. Ningún parámetro se modifica al consultar la prueba.

La selección compara las familias dummy, regresión logística, random forest e histogram gradient boosting. La configuración se elige en validación mediante precisión al umbral 0,85 con al menos tres señales. Después se calibra con sigmoide y se mide

sobre el bloque de prueba. Además, se ejecuta una ablación de la regresión logística sin retardos, cambios, retornos ni medias móviles de uno, tres y siete días. Esta comparación responde a una pregunta concreta: si el histórico añade información o solamente complejidad.

Configuración	ROC-AUC	Brier	Precisión @0,85	Señales
Logística con histórico	0,641	0,075	0,929	85
Logística sin histórico	0,760	0,072	0,970	67
Random forest con histórico	0,742	0,074	0,942	86
HGB con histórico	0,649	0,082	0,915	82

Tabla 6.4: Exploración inicial de familias en el corte temporal de marzo.

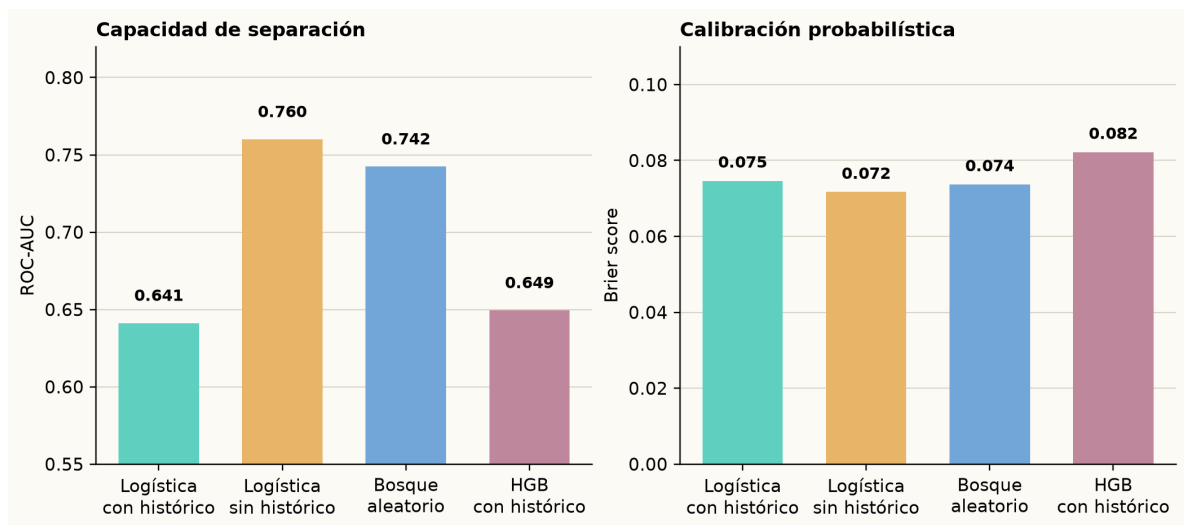


Figura 6.1: Exploración inicial de modelos y variables temporales en marzo. Un Brier menor indica mejor calibración.

La exploración inicial señala que la regresión logística sin características históricas puede separar mejor las dos clases que las alternativas probadas. También muestra que los retardos y medias móviles no deben darse por útiles sin contrastarlos. La siguiente ablación vuelve a medir esa decisión junto con la aumentación, usando particiones por fecha para que una fecha completa no se divida entre lados de un pliegue.

6.4.4 Ablación de histórico y aumentación

La segunda exploración usa una matriz 2×2 con regresión logística. Se mantienen los mismos cortes temporales de marzo, el umbral de selección 0,85, tres pliegues temporales y calibración sigmoide. Se comparan dos decisiones de diseño: conservar o retirar retardos y medias móviles, y entrenar con o sin aumentación numérica.

La aumentación se aplica únicamente al entrenamiento. Para cada variable numérica se genera una copia perturbada mediante

$$x'_k = x_k + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, 0,01 \cdot IQR(x_k)).$$

Las etiquetas, las variables categóricas y las fechas no cambian. Además, todas las filas de una misma fecha se mantienen dentro del mismo lado de cada pliegue temporal. De este modo la copia perturbada no puede aparecer en entrenamiento mientras su original está en validación. La validación y la prueba permanecen sin modificar.

Histórico	Aumentación	ROC-AUC	Brier	Precisión @0,85	Señales
Sí	No	0,631	0,076	0,931	87
No	No	0,716	0,076	0,952	62
Sí	Jitter	0,575	0,085	0,924	92
No	Jitter	0,613	0,084	0,919	86

Tabla 6.5: Ablación logística en el corte temporal de marzo.

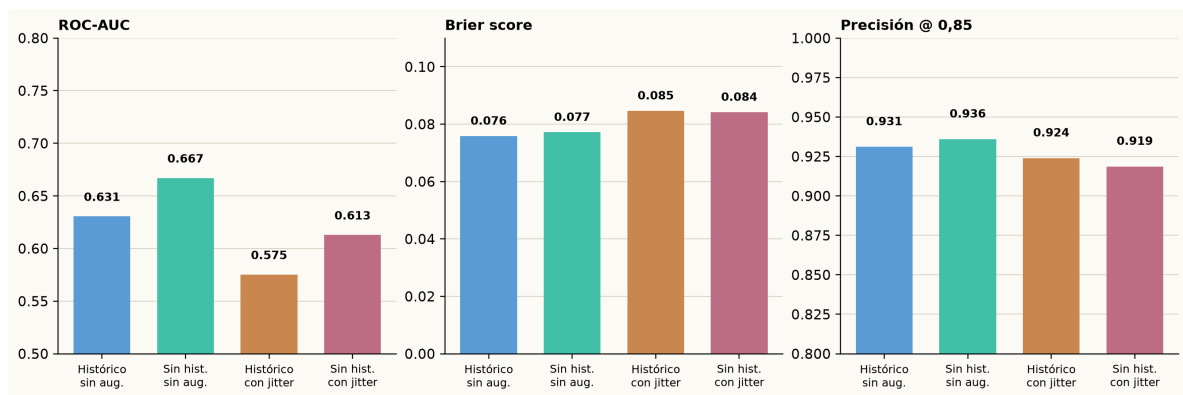


Figura 6.2: Efecto del histórico y de la aumentación numérica sobre la prueba de marzo.

La combinación sin histórico ni aumentación obtiene el mejor equilibrio entre separación y calibración. El jitter duplica el tamaño de entrenamiento de 171 a 342 filas, pero empeora ROC-AUC y Brier en ambas configuraciones. Por ello no se incorpora al modelo de referencia. Este resultado no prueba que la aumentación sea perjudicial en cualquier muestra. Indica que, con una colección corta y muy concentrada en operaciones positivas, introducir pequeñas variaciones de precio añade más ruido que información.

La lectura debe mantenerse prudente. Los 19 artículos disponibles en el corte aparecen tanto en entrenamiento como en prueba y la etiqueta positiva representa el 89,5 % de la prueba. Una estrategia que marca muchos casos como favorables puede alcanzar precisión elevada por la propia distribución de la muestra. El corte de mayo confirma este problema: de 361 ejemplos de entrenamiento, 76 de validación y 95 de prueba, las dos últimas particiones contienen exclusivamente etiquetas positivas. En esas condiciones ROC-AUC no es una métrica definida y no se entrena un modelo para presentarlo como comparación válida. Este resultado establece una condición de continuación clara: ampliar la cobertura cruzada BUFF–Steam y equilibrar los eventos antes de convertir el modelo en recomendación operativa.

6.4.5 Evidencia de pruebas automatizadas

La evidencia técnica se vuelve a ejecutar sobre el repositorio restaurado el 10 de agosto de 2026. La suite completa obtiene 276 pruebas superadas en 26,85 segundos. Incluye 273 pruebas unitarias y tres pruebas de integración contra la base de datos configurada. Las integraciones registran un snapshot de mercado, comprueban la actualización de una cotización y guardan una señal de oportunidad vinculada a un artículo. Cada una abre una transacción externa y la revierte al terminar, por lo que la comprobación no añade datos de prueba al histórico utilizado por los experimentos.

Bloque	Pruebas	Alcance
Unitarias	273	Economía, OCR, conectores, scraping, datasets, simulador, MARL y panel web.
Integración	3	Persistencia de snapshots, actualización de precios y señales en PostgreSQL con reversión.
Suite completa	276	Resultado: todas las pruebas superadas.

Tabla 6.6: Ejecución de pruebas automatizadas recuperada.

6.4.6 Dataset de trading operativo

El dataset `trading_profit_v1` se ha generado desde `market_history_points` usando precios Steam/BUFF, comisiones y horizonte temporal. La primera versión ha producido 2.331 ejemplos y 50 artículos, pero con desbalance extremo en validación y test: solo un caso positivo en cada split para la dirección analizada. Por este motivo, las métricas de ese entrenamiento no se usan como evidencia de rendimiento operativo.

El análisis posterior ha mostrado que el histórico de BUFF ha empezado a estar mejor alineado tras corregir el parser y permitir backfill de puntos históricos. Aun así, la dirección más natural de arbitraje, comprar en BUFF y vender en Steam, todavía no genera ejemplos suficientes a ocho días porque falta histórico futuro Steam alineado con los puntos BUFF recientes.

A partir de esta limitación, la validación distingue entre aprendizaje temporal e inferencia operativa: Steam se usa como fuente principal para aprender el riesgo temporal de salida y BUFF queda como precio vivo de entrada en inferencia. El objetivo no es predecir el precio exacto de BUFF. La recomendación evalúa si un precio BUFF puntual conserva margen suficiente frente a una salida Steam estimada y sus comisiones.

6.4.7 Dificultades y pivotaciones

Durante el desarrollo se han producido varias decisiones de pivotaje. La primera aparece al migrar el Excel operativo: el valor principal no estaba en copiar una hoja de cálculo, sino en extraer reglas económicas verificables y convertirlas en simulación reproducible. Esta decisión desplaza el trabajo desde una herramienta manual hacia un sistema auditable.

La segunda pivotación afecta a las fuentes de datos. El planteamiento inicial intentaba alinear históricos de Steam y BUFF para entrenar directamente sobre oportunidades de arbitraje entre plataformas. Las primeras pruebas mostraron dos problemas: falta de

histórico alineado suficiente y riesgo operativo al consultar BUFF de forma reiterada. Por ello, el enfoque se ajusta: Steam se mantiene como fuente principal de aprendizaje temporal y BUFF se usa de forma puntual como precio vivo de entrada para evaluar flips concretos.

La tercera pivotación aparece en la parte multiagente. Antes de ampliar entrenamiento y episodios, se prioriza que el entorno, las recompensas, restricciones y métricas sean comprobables. Por eso el estado actual se presenta como smoke funcional y no como validación definitiva de MAPPO. Esta decisión evita basar conclusiones en un entrenamiento que todavía depende de más datos, más episodios y comparaciones contra baselines.

6.4.8 Resultados de la arquitectura multiagente

El entorno multiagente se ha comprobado con el mismo dataset de marzo. Scout marca una oportunidad, Trader propone la compra de una unidad y Portfolio acepta o rechaza según saldo, exposición, liquidez y capital bloqueado. Una compra requiere el acuerdo de los tres roles. La recompensa compartida combina margen, inactividad, penalización por restricciones, drawdown y capital inmovilizado:

$$r_t = w_m \rho_t \mathbb{I}_{\text{compra}} - \lambda_i \mathbb{I}_{\text{oportunidad ignorada}} - \lambda_r v_t - \lambda_d DD_t - \lambda_b B_t.$$

En la ecuación, ρ_t es el retorno estimado de la oportunidad, v_t el número de restricciones incumplidas, DD_t el drawdown y B_t la proporción de capital bloqueado. La misma recompensa se entrega a los tres agentes para favorecer coordinación en lugar de optimización aislada.

Configuración sobre prueba	Compras	Efectivo final	Lectura
Política manual de margen positivo	13	309,64 EUR	Abre posiciones mientras el límite de riesgo lo permite.
Política de mantener	0	1.000,00 EUR	Baseline sin exposición.
Margen positivo sin probabilidad supervisada	13	309,64 EUR	Misma decisión, porque este dataset aún no incluye probabilidades por fila.
PPO multiagente, una iteración sobre entrenamiento	0	100,00 EUR	Smoke de integración, sin política aprendida utilizable.

Tabla 6.7: Pruebas iniciales de las políticas MARL.

La primera política es una regla escrita a mano que compra si el margen actual es positivo. No representa una política entrenada, pero comprueba la coordinación entre los tres roles y las restricciones de Portfolio. La política de mantener confirma el comportamiento contrario. En ambas pruebas se partió de 1.000 EUR y se recorrieron las 95 observaciones del bloque de prueba. Las posiciones quedan abiertas por el bloqueo temporal, por lo que la tabla no debe interpretarse como beneficio final.

El comando PPO con RLlib completó una iteración y 64 pasos de entorno con el

conjunto de entrenamiento. El estado central se expone al adaptador y hay una política por rol, pero el crítico centralizado queda pendiente. La evaluación del checkpoint corto no ejecutó compras. Este resultado es informativo: la compatibilidad técnica está validada, pero una única iteración no produce una política que pueda compararse económicamente con una regla de margen.

6.5 Experimentos finales pendientes

Para cerrar la validación del TFM será necesario ejecutar una fase experimental más estricta. Primero debe validarse el modelo Steam-only de salida a ocho días y conectarlo a candidatos con precio BUFF vivo. Después deben compararse baselines simples: comprar todos los candidatos de SteamDT, comprar solo cuando el margen BUFF $\rightarrow$ Steam es positivo, agente único y política dummy.

La evaluación MARL deberá incluir ablations. Las más relevantes son: sin probabilidad supervisada, sin agente Portfolio, sin penalización por capital bloqueado y sin restricciones de liquidez. Estas comparaciones permitirán distinguir si la mejora viene de la arquitectura multiagente, del modelo supervisado, de una regla de riesgo o simplemente de la estructura del simulador.

6.6 Discusión

6.6.1 Casos favorables

El trabajo realizado muestra que el sistema ya puede integrar fuentes, construir datasets, entrenar modelos supervisados, calibrar probabilidades, simular reglas económicas y ejecutar un smoke multiagente. La separación entre adquisición, predicción, simulación y decisión ha resultado útil porque permite avanzar por partes y detectar bloqueos concretos sin rehacer toda la arquitectura.

6.6.2 Limitaciones observadas

La principal limitación actual es la disponibilidad de histórico alineado entre Steam y BUFF. Sin suficientes ejemplos positivos en validación y test, cualquier métrica de trading puede ser engañosa. También existe riesgo de sobreajuste en variables categóricas de alta cardinalidad y en señales de precisión alta con pocos casos. Por último, el entorno MARL necesita una validación experimental más amplia antes de poder concluir si la arquitectura multiagente mejora realmente a una estrategia simple.

6.6.3 Lectura de los resultados

La lectura correcta de los resultados actuales es prudente. El proyecto ya demuestra viabilidad técnica de integración y simulación, pero todavía no demuestra superioridad económica de una política multiagente. Esta diferencia es importante para que la memoria sea defendible: una arquitectura ejecutable es una contribución técnica, mientras que la mejora de rendimiento requiere evidencia experimental adicional.

Conclusiones

7.1 Conclusiones principales

El desarrollo realizado hasta el momento permite concluir que el problema no puede reducirse a una predicción aislada de subida o bajada de precio. En mercados como el de *Counter-Strike 2*, la decisión depende de fricciones muy concretas: comisiones, conversión de moneda, liquidez, *trade hold*, capital bloqueado y riesgo de cartera. Por ello, la arquitectura propuesta necesita combinar adquisición de datos, simulación económica y decisión multiagente, siguiendo la disciplina de backtesting y control de fricciones propia de los entornos de trading con aprendizaje automático [2].

También se ha comprobado que la separación por capas facilita el avance del proyecto. Los agentes extractores se encargan de observar el mercado, el modelo supervisado produce señales probabilísticas, el simulador aplica reglas económicas y los agentes MARL operan dentro de un entorno controlado. Esta división reduce el acoplamiento y permite validar cada parte por separado.

La contribución principal de esta fase no es afirmar que el sistema gane dinero, sino construir una base reproducible para estudiar esa pregunta. El proyecto ya dispone de una arquitectura modular, un flujo de adquisición, un modelo de persistencia, un simulador económico, datasets derivados, modelos supervisados preliminares y un entorno multiagente funcional para smoke tests.

Una conclusión metodológica relevante es que el proyecto ha evolucionado a partir de las dificultades encontradas. La falta de histórico Steam/BUFF suficientemente alineado, el riesgo de scraping reiterado y el desbalance de los primeros datasets de trading han llevado a separar entrenamiento e inferencia operativa. Esta pivotación no cambia el objetivo del sistema, pero sí mejora su viabilidad: el aprendizaje temporal se apoya en Steam y la comparación con BUFF se reserva para evaluar oportunidades concretas.

7.2 Cumplimiento de objetivos

Se ha avanzado en la mayoría de objetivos técnicos: existe un monorepo modular, un modelo de datos operativo, pipelines de adquisición, análisis del Excel original, construcción de datasets, entrenamiento supervisado, simulador de cartera y un entorno MARL funcional para pruebas cortas. También se han definido agentes Scout, Trader y Portfolio, junto con restricciones de riesgo y métricas de evaluación.

El cumplimiento experimental todavía es parcial. Los resultados supervisados muestran que el pipeline funciona y que algunos modelos producen señales de alta precisión en umbrales exigentes, pero el número de señales es limitado. El dataset operativo de trading Steam/BUFF está construido, aunque su primera versión presenta desbalance extremo en validación y test. El entorno MARL ejecuta, pero no constituye todavía

una evaluación comparativa completa. Las pruebas automatizadas y los smoke tests de rendimiento forman parte del capítulo experimental porque verifican que las métricas obtenidas proceden de componentes comprobados.

7.3 Limitaciones

La limitación más importante es la escasez de histórico alineado entre plataformas para algunas direcciones de trading. El dataset más cercano al objetivo operativo presenta pocos positivos en validación y test, por lo que no permite defender todavía una mejora robusta. Además, algunos resultados supervisados tienen alta precisión en umbrales altos, pero con pocas señales, lo que obliga a interpretarlos con prudencia. Por ello, se separa el aprendizaje temporal basado en Steam del uso puntual de BUFF como precio vivo de entrada para flips.

Otra limitación es que el smoke multiagente demuestra que el flujo ejecuta, pero no que el sistema haya aprendido una política superior. Para llegar a esa conclusión se necesita una evaluación reproducible con más episodios, más datos, baselines justos y métricas de riesgo.

También existe una limitación propia del dominio. Los activos de CS2 dependen de plataformas externas, reglas de intercambio, sesiones, cambios visuales y restricciones que pueden variar. Por tanto, cualquier sistema automatizado debe mantener trazabilidad, parámetros versionados y capacidad de revisar errores de fuente.

7.4 Trabajo futuro

Las líneas inmediatas de trabajo son validar un modelo Steam-only de salida a ocho días, usar BUFF como precio vivo de entrada en inferencia de flip, comparar contra baselines simples y entrenar MARL con una configuración CTDE completa. Después será necesario estudiar ablations, generar gráficas de resultados y cerrar una discusión experimental sobre cuándo la arquitectura multiagente aporta valor y cuándo no.

Los experimentos finales deberán reportar beneficio neto, saldo final, drawdown, capital bloqueado, número de operaciones, posiciones abiertas y rechazos por riesgo. Además, deberán comparar la arquitectura completa contra versiones simplificadas: sin probabilidad supervisada, sin agente Portfolio y con reglas simples de spread. Solo con esa comparación será posible defender si la descentralización multiagente aporta una mejora frente a estrategias más sencillas, siguiendo la lógica comparativa habitual en MARL y entrenamiento cooperativo [5, 6].

Bibliografía

- [1] Vili Lehdonvirta. Virtual item sales as a revenue model: Identifying attributes that drive purchase decisions. *Electronic Commerce Research*, 9(1):97–113, 2009.
- [2] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the Second ACM International Conference on AI in Finance*, 2021.
- [3] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [5] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.
- [6] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative, multi-agent games. In *Advances in Neural Information Processing Systems*, 2022.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. <https://arxiv.org/abs/1707.06347>, 2017.
- [8] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [9] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [10] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632, 2005.
- [11] Justin K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Rodrigo Perez, Caroline Horsch, Clemens Diefendahl, Niall L. Williams, Yashas Lokesh, Praveen Ravi Hines, and Niall Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2021.
- [12] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.

Anexos

A.1 Estructura del repositorio

El repositorio se organiza como un monorepo. La carpeta **apps** contiene aplicaciones y comandos ejecutables. La carpeta **packages** contiene la lógica reutilizable. La carpeta **docs** recoge decisiones y documentación técnica. La carpeta **tests** contiene pruebas unitarias e integración. La carpeta **TFM** mantiene la memoria en L^AT_EX.

Los módulos principales son:

- **apps/acquisition**: extractores, scraping, OCR, importadores y SteamDT.
- **apps/cli**: comandos de construcción de datasets, entrenamiento, scraping y evaluación.
- **apps/web**: interfaz web local de consulta y control operativo.
- **packages/contracts**: contratos Pydantic compartidos.
- **packages/persistence**: conexión, repositorios y persistencia.
- **packages/prediction**: entrenamiento e inferencia supervisada.
- **packages/simulation**: reglas económicas, cartera, riesgo y backtesting.
- **packages/marl**: entorno multiagente, wrappers PettingZoo/RLlib y entrenamiento.
- **packages/vision**: OCR, preprocesado y parsing visual.

A.2 Configuración del entorno

El entorno se define en `pyproject.toml`. El proyecto usa Python, Supabase/Postgres, Pytest, Ruff, Mypy y dependencias opcionales para MARL como Ray/RLlib, PettingZoo, Gymnasium y PyTorch. Para desarrollo local existe un `docker-compose.yml` con Postgres.

Comandos representativos:

```
python -m pytest tests/unit
python -m ruff check
python -m mypy apps packages tests/unit
python -m apps.cli.auto_scrape_loop --once
python -m apps.cli.render_stream_scrape
python -m apps.cli.web_dashboard_server
```

A.3 Comandos de datasets y modelos

Los comandos de datos y modelos separan adquisición, construcción de dataset, entrenamiento e inferencia. Esta separación permite repetir una fase sin rehacer todo el flujo.

```
python -m apps.cli.build_supervised_dataset
python -m apps.cli.analyze_supervised_coverage
python -m apps.cli.train_supervised_model
python -m apps.cli.build_trading_dataset
python -m apps.cli.retrain_trading_model
python -m apps.cli.score_live_opportunities
```

A.4 Parámetros de entrenamiento

Los experimentos supervisados usan splits temporales, calibración de probabilidades y selección por precisión operativa en umbrales altos. En los smoke tests se han comparado modelos dummy, regresión logística, random forest e histogram gradient boosting. El entrenamiento MARL usa RLlib con un entorno multiagente y tres políticas: Scout, Trader y Portfolio.

Bloque	Parámetros relevantes
Supervisado	Split temporal, umbral operativo, calibración, modelo candidato y columnas excluidas por leakage.
Trading	Dirección de arbitraje, horizonte, fees, tipo de cambio, ventana histórica y target de beneficio neto.
Simulación	Saldo inicial, tamaño máximo de posición, <i>trade hold</i> , comisiones y límites de exposición.
MARL	Número de episodios, políticas Scout/Trader/Portfolio, algoritmo PPO y métricas de cartera.

Tabla A.1: Parámetros a versionar en los experimentos.

A.5 Modelo de datos

El modelo operativo actual gira alrededor de dos tablas principales: `market_items` y `market_history_points`. La primera representa variantes reales de artículos y su estado actual por plataforma. La segunda almacena series temporales en formato largo, con una fila por artículo, plataforma, fecha y métrica. Esta estructura evita mezclar métricas distintas y permite añadir nuevas señales sin alterar la forma general del histórico.

El esquema canónico previsto añade entidades como `assets`, `platforms`, `market_observations`, `predictions`, `agent_actions`, `investment_decisions` y `simulated_positions`. El objetivo de este esquema es mejorar trazabilidad, auditoría y separación entre observación, predicción y decisión.

A.6 Resultados adicionales

Los resultados intermedios se guardan en `model-runs` y los datasets derivados en `data/datasets`. Estas carpetas permiten conservar reportes de cobertura, exploración de características, métricas de entrenamiento, umbrales seleccionados y salidas de smoke tests sin mezclar los artefactos pesados con el código principal.

Para que un experimento pueda incorporarse como evidencia final en la memoria, debe registrar como mínimo: configuración usada, periodo temporal, número de activos, tamaño de cada split, tasa de positivos, modelo o política evaluada, métricas principales y limitaciones observadas.